

Gratitude - Server configuration guide

Process for the server configuration team

project_grat

September 7, 2026

Purpose

Use this document as the setup process for production (or shared staging). Follow the steps in order. The application team delivers the code. Your team provisions hosts, networking, TLS, Postgres, disk and process supervision.

What you are configuring

- **Database host** - PostgreSQL 16 (14+ OK).
- **Application host** - Node.js 20+, nginx (HTTPS), Node API, static web files and a separate uploads disk.

Why a separate uploads disk

Card images and compiled HTML are stored on the application host under `api/uploads/`, not in PostgreSQL and not on cloud object storage yet.

- Without this folder, imported cards have no images.
- If the app server is rebuilt or the OS disk is replaced, a normal install disk wipe would delete every card asset.
- Email and the UI load images from this path via the public API URL (`/uploads/...`).

Therefore mount a **persistent volume** (50 GB minimum recommended) at the uploads path and include it in backup/DR alongside the database.

Sizing (minimum)

Host	CPU / RAM	Disk
Database	2 vCPU / 4 GB	40 GB SSD for Postgres data
Application	2 vCPU / 4 GB	40 GB OS + 50 GB persistent uploads volume

Ports

- **443** - HTTPS to the application host (intranet).
- **3001** - API on localhost or private only. nginx proxies to it (do not expose publicly).
- **5432** - Postgres. Allow only from the application host.

Configuration process

1. Database host

1. Install PostgreSQL 16.
2. Create an empty database (e.g. `project_grat`) and a least-privilege role.
3. Restrict port 5432 to the application host only.
4. Enable nightly backups (retain at least 7 days).
5. Give the application team the connection string for `DATABASE_URL`.

2. Application host - base

1. Install Node.js 20+ and nginx.
2. Attach and mount the persistent uploads volume (see section above).
3. Place the application release on the host (or pull from the repo).
4. Open inbound 443 from the intranet. Keep 3001 private.

3. Application secrets and env

Create `api/.env` from `api/.env.example`. At minimum set:

- `DATABASE_URL` - Postgres connection string from step 1.
- `JWT_SECRET` - strong random secret (not the development default).
- `CORS_ORIGIN` - real HTTPS web origin(s).
- `PUBLIC_API_URL` - public HTTPS API origin (used for card image URLs in email).

Full optional mail/Azure variables are listed in `api/.env.example`.

4. Install, migrate and start API

1. Run `npm ci` in `api/`.
2. Run `npx prisma generate` then `npx prisma migrate deploy`.
3. Do **not** run seed on production unless deliberately resetting demo data.
4. Start the API under systemd (or equivalent): `npx tsx src/index.ts`.
5. Confirm `GET /health` returns OK.

5. Build and serve the web UI

1. Run `npm ci` in `web/`.
2. Build with `VITE_API_URL` set to the public HTTPS API URL.
3. Point nginx at `web/dist` (SPA fallback to `index.html`).
4. Proxy API traffic to port 3001. Ensure `/uploads/` is reachable on the public API origin.
5. Install the TLS certificate on nginx.

6. Handover checks

1. Browser opens the site over HTTPS.
2. Login works (today: development login only. Azure AD is not ready yet).
3. A test upload appears under the uploads volume and loads in the browser.
4. Database and uploads backups are scheduled and verified.

Notes for your team

- No Docker or sample nginx unit is in the repo - write nginx + systemd from this guide.
- Azure AD login is not production-ready yet (AUTH_MODE=dev only).
- Treat the uploads volume as critical data, same as the database.